scientific reports

Check for updates

OPEN

Network-based intrusion detection using deep learning technique

Muhammad Farhan[1], Hafiz Waheed ud din[1], Saadat Ullah[1], Muhammad Sajjad Hussain[1], Muhammad Amir Khan[2]✉, Tehseen Mazhar[3,6]✉, Umar Farooq Khattak[4] & Ines Hilali Jaghdam[5]

A high growth rate in network traffic and the complexity of cyber threats have made it necessary to create more effective and flexible intrusion detection systems. Most traditional Network-based Intrusion Detection Systems (NIDS) can become weak at detecting new patterns of attacks due to the use of obsolete data or traditional machine learning models. To overcome the mentioned constraints, the current research presents a new deep learning solution that combines Sequential Deep Neural Networks (DNN) and Rectified Linear Unit (ReLU) activation unit with an Extra Tree Classifier feature selection procedure. The proposed model is trained and tested on the new rich and up-to-date UNSW-NB15 set, which provides a realistic reflection of the real-life network traffic and attack vectors. The interesting novelty of this study is the tactical use of ReLU-based DNN combined with feature optimization through the Extra Tree Classifier, which not only overcomes general problems like vanishing gradients and overfitting but also greatly increases the interpretability of the model and the efficiency of its computation. This dimensional reduction of the feature space (43 to only 8 highly relevant features) retains the high accuracy of the model but with better inference speed, which is a crucial aspect of the real-time deployment of NIDS. The results show that with the Sequential DNN approach, the binary class (0 for normal and 1 for attack records) achieved 97.93% accuracy, 97% Precision, 97% Recall and 97% F1-score. Furthermore, the detailed experimental testing, such as ROC curves and Confusion Matrices, confirmed that the Sequential DNN performed well in comparison to other Existing Studies. These findings underscore the effectiveness of deep learning architectures enhanced with optimized feature selection in detecting network intrusions, making the proposed system a promising solution for securing critical infrastructure in sectors such as finance, healthcare, and government networks.

Keywords Network attacks, Network-based intrusion detection system (NIDS), Deep neural networks, Extra tree classifier, Activation function

Due to cyber-attacks that are becoming more common and advanced, IDS plays a vital role in contemporary cyber security systems because organizations require effective ways to monitor their networks and protect themselves against all kinds of cyber-attacks[1,2]. Conventional security solutions like firewalls are ineffective at identifying and stopping sophisticated threats, particularly zero-day attacks and abnormal activities. This brings an imperative necessity in the development of smart solutions such as IDS that will examine network traffic patterns and detect malicious behaviors on a real-time basis. This study aims to make use of Deep Learning (DL) techniques because of their demonstrated effectiveness at analyzing large amounts of data via high-dimensional, high-complexity data such as network traffic logs. Compared with classical machine learning-based models that require extensive exploitation of experts to engineer features, deep learning models can learn a hierarchical feature representation of raw data, which permits more precise and semi-dynamic intrusion identifications.

[1]Department of Computing and Information Technology, Faculty of Computing, Gomal University, Dera Ismail Khan 29050, Pakistan. [2]School of Computing Sciences, Faculty of Computer Science and Mathematics, Universiti Teknologi MARA, 40450 Shah Alam, Selangor, Malaysia. [3]School of Computer Science, National College of Business Administration and Economics, Lahore 54000, Pakistan. [4]School of Information Technology, UNITAR International University, Kelana Jaya, Petaling Jaya 47301, Malaysia. [5]Department of Computer Science and Information Technology, Applied College, Princess Nourah bint Abdulrahman University, P.O. Box 84428, Riyadh 11671, Saudi Arabia. [6]Department of Computer Science and Information Technology, School Education Department, Government of Punjab, Layyah 31200, Pakistan. ✉email: amirkhan@uitm.edu.my; tehseenmazhar719@gmail.com

Content courtesy of Springer Nature, terms of use apply. Rights reserved

Additionally, DL models, particularly Deep Neural Networks (DNNs), present higher performances in detecting attacks with low and previously unknown patterns because of their non-linearity learning abilities and resilience. To support these motives, we provide a new framework, which is a combination of a Sequential Deep Neural Network and ReLU activation functions and Extra Tree-based feature selection with the modern UNSW-NB15 record. This method is a response to computation efficiency and detection performance, increasing performance metrics compared to the benchmark models. There are two types of Intrusion Detection Systems: Host-based and Network-based. A Host-based IDS (HIDS) is a security tool often deployed throughout a network and installed on individual hosts or endpoints. It monitors hosts' activities and identifies potentially harmful or suspicious behavior. The operation of HIDS software involves tracking the action on the host device itself to search for indications of unauthorized access, malicious code, or other security concerns. For instance, if a user attempts to access a file for which they do not have permission, the HIDS software can alert security staff to investigate and respond accordingly. A HIDS is a powerful tool for detecting and responding to security risks on individual hosts or endpoints, and it can help companies enhance their overall security posture. A Network-based IDS (NIDS) that monitors and evaluates network traffic is a security tool that assists in identifying possible security issues and taking the necessary steps to fix them. The operation of NIDS software involves monitoring network data and then producing warnings or taking measures in response to any possible vulnerabilities. A wide range of security risks, including malware, DoS attacks, and other types of malicious activity, can be detected by NIDS systems. Figure 1 represents a network security system presenting both a host-based Intrusion Detection System (HIDS) and a network-based IDS. Presenting the setup of firewalls, a router, and a WWW server for managing, monitoring and controlling network traffic for security reasons.

Likewise, in recent times, research related to deep learning-based Intrusion Detection has been carried out using different datasets such as KDDCUP99, KDD98, and NSL-KDD7[4]. Nonetheless, these datasets influence the results of intrusion systems due to the absence of modern-day traffic data. Henceforth, enhancing the functionality of the current NIDS feels the necessity of adapting modern and state-of-the-art datasets. In this research work, a UNSW-NB15[5] dataset-based framework has been developed for the detection of attacks in the network by using a deep learning technique[6,7]. Here are the key novelties of research:

- Propose a new combination of a Sequential Deep Neural Network (DNN) with Rectified Linear Unit (ReLU) activation and an Extra Tree Classifier feature selection to the UNSW-NB15 dataset. Previous works have considered DNNs but with conventional activation functions, such as sigmoid or tanh, which have the drawback of the vanishing gradients problem and poor behavior with deeper networks.
- Apply the Relu Activation function to mitigate vanishing gradients problems.
- Apply Extra Tree Classifier as a Feature Selection Technique: Dimensionality reduced to 8 important features out of 43 features, enhancing the computation capability without compromising accuracy.
- Achieved high-performance metrics in binary classification, including an accuracy of 97.93%, along with 97% precision, recall, and F1-score, surpassing previous benchmark studies.

The structure of the paper is as follows: The related works that are similar to the current study are presented in Section II. Section III discusses the methodology for the development of the framework. The results and discussion are presented in Section IV. While in the end, the conclusions are presented in Section V.

Related work

Deep learning-based intrusion detection systems have been the subject of extensive analytics research over the last few years. Due to its capacity for growth and learning, this field of study is gaining popularity. Because of this, it is very successful and highly effective in addressing an alarming rise in unknown attacks. The use of a deep learning approach on the KDD-99 dataset to create a deep learning method for finding anomalies has been analyzed by[8–11]. Since the UNSW-NB15 data set is complicated[12], so according to the experimental findings, UNSW-NB15 was more complicated than the dataset of KDD99 and is now regarded as a new benchmark data set to evaluate NIDS. In[13], an improved way of using vision-based deep learning models to process network

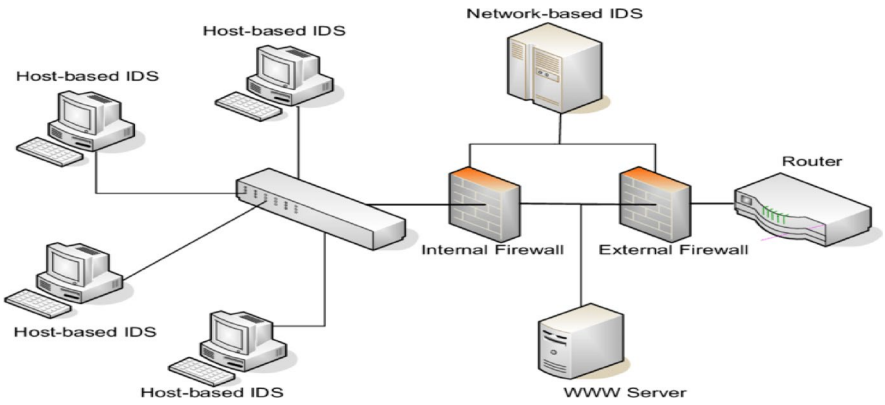


Fig. 1. Host-based IDS and network-based IDS[3].

Content courtesy of Springer Nature, terms of use apply. Rights reserved

intrusion detection systems is presented. While a multiclass Artificial Neural Network (ANN) for intrusion detection in computer networks has been introduced[14]. Whereas, a deep neural network approach was evaluated on the datasets of KDDCUP 1999 and UNSW-NB15. The results of tests show that the model may be able to find cyberattacks in computer networks. Similarly, the application and analysis of DNN as a deep learning model to identify and categorize an adaptive and efficient intrusion detection system has been carried out[15]. While, a two-stage classifier based on a subset of protocols and the RepTree algorithm for network IDS has been presented in[16]. The datasets of UNSW-NB15 and the NSL-KDD were used in the study to test the effectiveness of the classifier. The results of the experiments showed that the detection accuracy on the complete datasets of UNSW-NB15 and NSL-KDD was 89.85% and 88.95%, respectively. In[17], a two-stage strategy for network intrusion recognition was proposed. The approach was founded on the GoogleNet Inception model and the deep convolution organization (CNN) model.

According to the results, the strategy that was suggested for enhancing the consistent quality of NIDS was effectively presented. Using the dataset from NSL-KDD[18], analyzed three different algorithms of machine learning. In the study, the NSL-KDD dataset was pre-processed, the selection of ML classifiers (SVM, RF, and ELM) was carried out, and accuracy, precision, and recall were measured. In[19], a dataset from KDDCUP99 was utilized. The study proposed a decision tree-based IDS framework in a big data over fog environment. The findings demonstrated that the suggested approach was both reliable and efficient. While[20] examined the benefits and drawbacks of contemporary machine and deep learning intrusion detection algorithms. The study[21] demonstrates an efficient incentive mechanism in Vehicular Networks that is simple but helps to select the agents having high-quality resources and to encourage them to act in the learning process so that the resulting aggregated model would be accurate, and the network overhead would be smaller. The paper[22] introduces a recently identified front-end Web attack that leverages the existing object storage services vulnerability of cloud providers to evade content security policy. The proposal of the study[23] is to introduce an unpacking system to the smart blockchain applications: AutoD, which is founded on the deception call on the JNI layer of Android ART. The study[24] propose a lightweight and simple to implement and augment unpacking framework named DeepAutoD (hereinafter referred to as d-ad) on the safety of distributed computers. The authors of the study[25] suggests an automated and consistent methodology of detecting BGP community attributes in AS-level. The propose study[26] an interactive gradient shielding (IGS) approach that first chooses sensitive areas and then uses gradient-based attack.

The article[27] suggests a new Network-based Intrusion Detection Machine Learning (NIDML) model, which is a learning model consisting of an ensemble of Decision Trees, Random Forests, K-nearest neighbors, Neural Networks, and Ensemble methods. The LUFlow dataset on which each of the algorithms was trained and tested ranges between 93.03% and 99.96% accuracy of the resulting models. In the paper[28], modern technical methods of improving the quality of work of intrusion detection systems are studied due to the critical importance of ensuring the security of computer networks against malicious activity. The model trains and validates the DNN-based IDS on the NSL-KDD dataset, which is a common benchmark in IDS research. The model trained and validates well with an accuracy of 91.30% and 94.38%, respectively. In this research[29], a Hybrid Deep-Learning-Based Network Intrusion Detection System (HDLNIDS) is used to build a hybrid intrusion detection system based on deep-learning to identify attacks across a network. To identify the effectiveness of the proposed system, the experiments are performed on publicly available benchmark CICIDS-2018 data. Simulation results of the research prove that the suggested HDLNIDS is more effective than the existing intrusion detection methods, having an average accuracy of 98.90% in identifying malicious attacks. The study[30] presents a deep learning-based detection framework to enhance the accuracy of threat detection by automatically extracting features and modeling nonlinearity. A detection model based on KDD Cup 1999 and UNSW-NB15 data sets supporting convolutional neural network (CNN), long short-term memory network (LSTM) and hybrid architecture designed to efficiently optimize preprocessing of data and design lightweight models in a systematic manner. It has been observed through experiments that the overall performance of the proposed CNN- LSTM integration model on the two types of datasets is much higher than the traditional machine learning and support vector machine models. F1 value is 0.9 (90%) of detecting KDD Cup 1999 data set and 0.86 (87%) of detecting UNSW-NB15 data set. The study[31] suggests a combined strategy employing the convolution neural network (CNN) and the long short-term memory (LSTM) with the hyper parameters optimization based on Bayesian optimization method in attacks detection in IoT. A novel stacked auto-encoder and soft-max classifier-based two-stage deep learning model for efficient network intrusion detection was proposed by[32]. Whereas the author[33] used the dataset of UNSW-NB15 as part of a two-step process for finding network intrusions based on anomalies. The researchers selected the dataset attributes that were most useful for ML using a variety of techniques, some of which include Recursive Feature Elimination and Random Forests. The implementation of a method for deep learning to automatically detect anomalies was studied in[34]. The experiment results demonstrated that the newly proposed deep learning model was better than the previous model in terms of performance.

A comparison of the performance of classical machine learning techniques was carried out in[35], which required significant feature engineering on the dataset of NSL-KDD. The research conducted using deep and machine learning-based network intrusion systems is still not sufficient. However, some of the related research work is summarized in Table 1.

Content courtesy of Springer Nature, terms of use apply. Rights reserved

Reference	Dataset (Complete/Partial)	Model	Accuracy	Research Gap
[12]	UNSW-NB15 (Partial)	ANN	Accuracy between 78.47% and 85.56%	Techniques for data Pre-processing have not been used in this research paper to improve accuracy further.
[36]	UNSW-NB15	ANN	Accuracy between 63.9% and 99.32%	The model is implemented on Binary Classification. The 80,000 data is selected for research.
[11]	UNSW-NB15	ANN	Accuracy: 92%	A good Binary classification accuracy has not been achieved due to class imbalance.
[32]	UNSW-NB15	Deep-stacked auto-encoder (DSAE) neural network	Accuracy:89.13%	The paper does not demonstrate how well the model recognizes minority attack classes.
[37]	UNSW-NB15	ANN	Accuracy between 53.43% and 82.66%	Due to a problem with class imbalance, the proposed model has not been able to attain good accuracy.
[38]	UNSW-NB15	Random Forest	Accuracy: 87.09%	The paper applies the Random Forest model to binary classes. The experiments concluded that a Random Forest cannot achieve good accuracy.
[27]	LUFlow dataset	Decision Trees, Random Forests, K-nearest neighbors, Neural Networks, and Ensemble methods	Accuracy: 99.96%	LUFlow dependency; no cross-dataset validation; unproven class balancing
[29]	CICIDS-2018	HDLNIDS	Accuracy: 98.90%	No reporting of **inference speed** (critical for real-time NIDS).
[28]	NSL-KDD	Deep Neural Network	Accuracy: 94.38%	Old Dataset
[6]	UNSW-NB15	Random Forest	Accuracy: 95.30%	Minimal Discussion on Class Imbalance
[7]	UNSW-NB15	LSTM	Accuracy: 92.2%	Reliance on Specific Dataset Features

Table 1. Summary of existing research Work.

The brief assessment of related work makes it clear that more research is needed to pinpoint the characteristics of network attacks. Henceforth, finding a general model with greater accuracy for the attacks in the dataset is necessary. In this work, the use of a sequential deep learning model with an extra tree classifier has been carried out to detect intrusion. Moreover, a dataset called UNSW-NB15 has been used. Furthermore, there has been limited research that has used DNN to detect intrusion using the under-consideration dataset.

Methodology

The development of an efficient deep-learning model for the identification of attacks within a dataset is of high significance. Therefore, this study is focused on utilizing DNN, a deep learning algorithm, to detect attacks considering the UNSW-NB15 dataset. The overall framework representing the proposed methodology is depicted in Fig. 2. The steps involved in the implementation of the DNN model in the NIDS are discussed below.

The following flowchart 2 shows the procedure involved in predicting the intruders based on the UNSW-NB15 dataset. Pre-processing, such as standardization, feature selection, and splitting data into training and testing sets, was then followed by using an Extra Tree Classifier and a sequential deep neural network model for prediction.

Unsw-Nb15 dataset

UNSW-NB15 is a network intrusion dataset that has 44 distinct features. In this dataset, there are 43 input variables and one output variable. The last column of the dataset describes the binary-class feature. These variables were used in the training process of the DNN model.

Data standardization

Data standardization sometimes referred to as data scaling, is an important data pre-processing method used in a variety of data analysis and deep learning activities. In the proposed work, data standardization aims to convert a dataset's features into a common scale (between 0 and 1) to improve the accuracy of the model. The relationship used to achieve standardization is given as:

$$z = x - \mu/\sigma \tag{1}$$

Feature selection techniques

Feature selection is a method for choosing features that primarily correlate with and contribute to the dataset's targeted variable. There are different techniques for feature selection, such as information gain attributes, correlation attributes, and principle component analysis[39]. However, in this framework, an extra tree classifier has been used. An extra tree classifier is a feature selection technique in which highly relevant features can be selected. After applying the feature selection technique, the data is ready for training the DNN model.

Content courtesy of Springer Nature, terms of use apply. Rights reserved

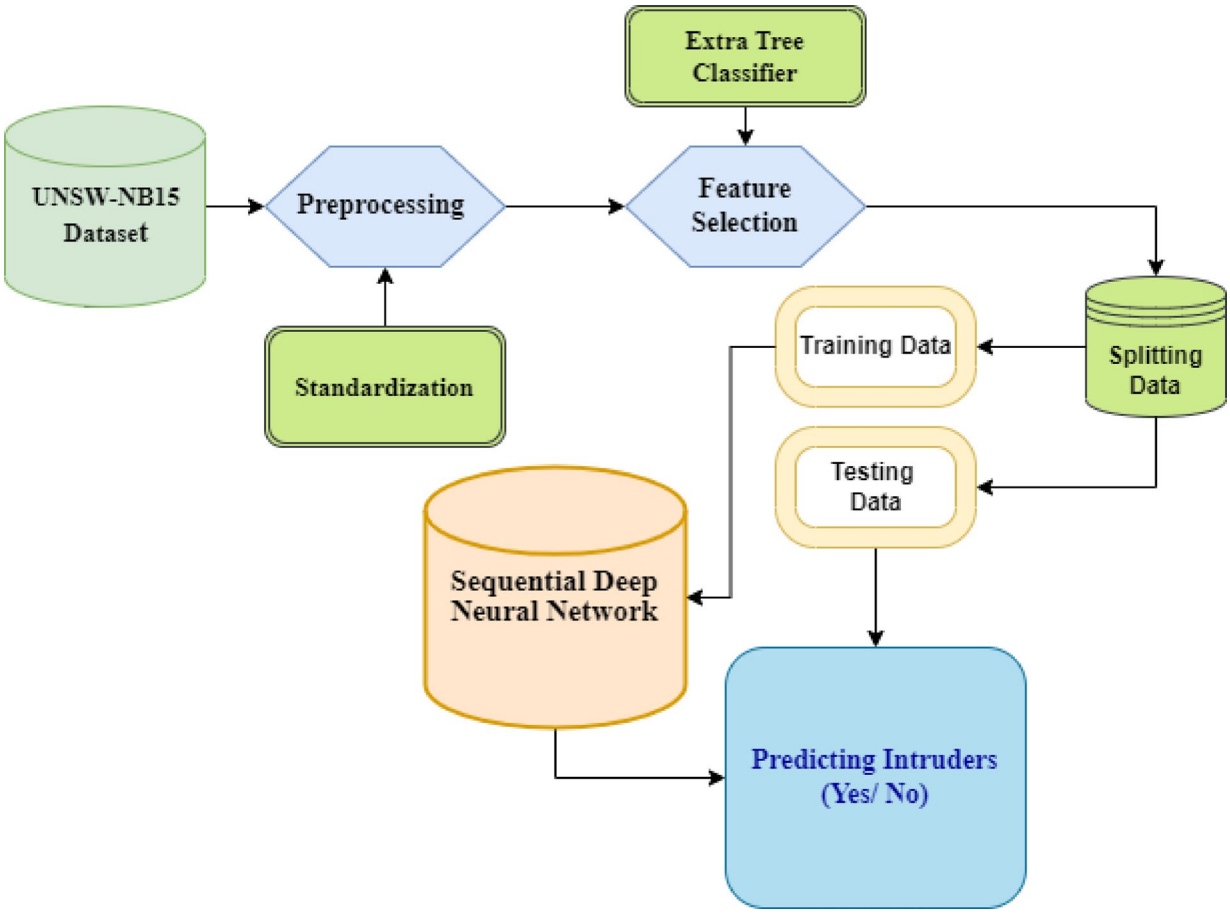


Fig. 2. Proposed model for network intrusion detection.

Deep neural network

An artificial neural network (ANN) that has several hidden layers in between the input and output layers is known as a deep neural network (DNN). Multiple neurons are present in each layer of a DNN, and these neurons are structured by one of three primary categories:

The input layer, which takes raw data or features from the dataset, is called the input layer. The hidden layer, with the presence of many hidden layers in a DNN, helps in evaluating the data input and learning to

Feature number	Feature/format	Feature number	Feature/format	Feature number	Feature/format
1.	Id(Integer)	16.	dloss(Integer)	31.	response_body_len(Integer)
2.	dur(Float)	17.	sinpkt(Float)	32	ct_srv_src(Integer)
3.	proto(Categorical)	18.	dinpkt (Float)	33.	ct_state_ttl(Integer)
4.	service(Categorical)	19.	sjit (Float)	34.	ct_dst_ltm(Integer)
5.	state(Categorical)	20.	djit (Float)	35.	ct_src_dport_ltm(Integer)
6.	spkts(Integer)	21.	swin(Integer)	36.	ct_dst_sport_ltm(Integer)
7.	dpkts(Integer)	22.	stcpb(Integer)	37.	ct_dst_src_ltm(Integer)
8.	sbytes(Integer)	23.	dtcpb(Integer)	38.	is_ftp_login(Binary)
9.	dbytes(Integer)	24.	dwin(Integer)	39.	ct_ftp_cmd(Integer)
10.	rate(Float)	25.	tcprtt (Float)	40.	ct_flw_http_mthd(Integer)
11.	sttl(Integer)	26.	synack (Float)	41.	ct_src_ltm(Integer)
12.	dttl(Integer)	27.	ackdat (Float)	42.	ct_srv_dst(Integer)
13.	sload(Float)	28.	smean(Integer)	43.	is_sm_ips_ports(Binary)
14.	dload(Float)	29.	dmean(Integer)	44.	label(Binary)
15.	sloss(Integer)	30.	trans_depth(Integer)		

Table 2. Features of dataset.

Content courtesy of Springer Nature, terms of use apply. Rights reserved

correctly represent valuable patterns based on the information they are given. The mathematically hidden layer is described as:

$$z_j = \sigma\left(\sum_{i=1}^{n} w_{ji}x_i + b_j\right) \tag{2}$$

In the hidden layer, there are different types of activation functions, such as ReLU, Sigmoid, and Tanh functions. The ReLU activation function has been used in this framework to introduce non-linearity in the DNN model. The mathematical equation for ReLU is:

$$f(x) = max(0, x) \tag{3}$$

1) Output Layer: following the processing of the information contained in the hidden layers, the output layer is responsible for producing the final predictions or outputs. The network's task determines the count of the output layer's neurons. In the case of a problem involving binary classification, for instance, there will be two output neurons reflecting the probabilities of belonging to each class, and in the case of a problem involving many classes, there will be several output neurons, one for each class. The mathematical equation for the output layer (binary classification) is.

$$\hat{y} = \sigma(Wx + b) \tag{4}$$

where:
ŷ: is the predicted output (a probability between 0 and 1).
σ: represents the sigmoid function, which is defined as: σ(z) = 1 / (1 + exp(− z)).
W: is the weight matrix connecting the last hidden layer to the output layer.
x: is the output of the last hidden layer.
b: is the bias term for the output layer.
The architectural diagram of Deep Neural Network is shown in Fig. 3.
The proposed work uses DNN models for binary class. In the binary class, there will be two output neurons.
The following hyperparameters were used in the proposed model by using grid search strategy.

Hyperparameter	Value
Input Layer	8 neurons (based on 8 features selected by Extra Trees).
Hidden Layer	Three dense layers with 800, 800, and 400 units respectively.
Output Layer	Two neurons for binary classification
Optimizer	Adam
Learning Rate	0.001 (Default value for Adam)

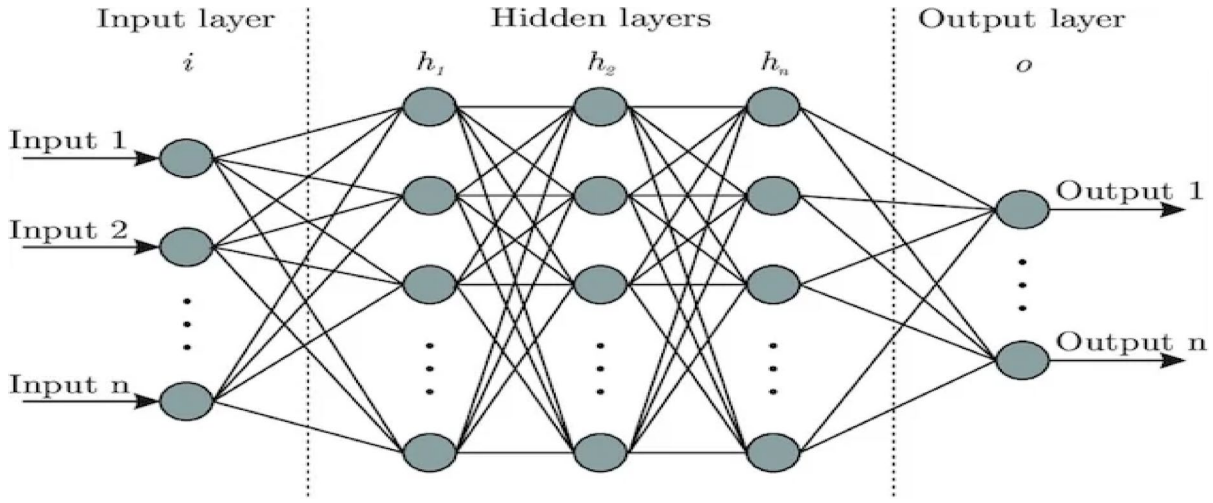


Fig. 3. Architectural diagram of deep neural network[40].

Content courtesy of Springer Nature, terms of use apply. Rights reserved

Hyperparameter	Value
Epochs	100
Batch Size	50
Loss Function	Categorical Crossentropy
Activation Function	Relu (use in Hidden Layer) Softmax (in the Output layer)

After performing the DNN model on the training dataset, the model will be predicted on the test dataset to evaluate the accuracy of the model. During the training of the model, certain issues and threats to the validity of the proposed work were observed, which are discussed in this section. A major issue during the training of the model was overfitting, which occurs when the model becomes highly specialized in learning from the training data and fails to generalize to new data. To resolve the problem of overfitting, regularization strategies such as L2 regularization, in conjunction with dropout, were utilized. Moreover, another major issue was data quality. To improve the data quality, careful pre-processing of the data, handling missing values, and removing duplicate records were carried out. During the training process of the proposed model, vanishing and exploding gradient issues were also faced, which were addressed by using an appropriate activation function, such as ReLU. Furthermore, hardware and computational limitations were also a concern, which degraded the computation of the proposed model. To solve this issue, hardware accelerators such as TPUs were used, whereas optimization of the code helped increase the speed of the training process.

Evaluation metrics

Some of the performance measures used to evaluate the models through the utilization of the confusion matrix are discussed below.

Accuracy shows the ratio of the number of data points that were correctly classified over the total number of data points[41]. Mathematically, accuracy is defined as follows:

$$Accuracy = \frac{TP + TN}{TP + TN + FP + FN} \tag{5}$$

Precision shows the ratio of true positive predictions as a percentage of all positive predictions. The following relationship is used for precision:

$$Precision = \frac{TP}{TP + FP} \tag{6}$$

Recall estimates the number of predictions that came true out of all the positive instances[42], where:

$$Recall = \frac{TP}{TP + FN} \tag{7}$$

Finally, the F1-Score shows the harmonic mean of recall and precision[43]. Mathematically,

$$F1 - score = \frac{2^{*}\left(Precision^{*}Recall\right)}{\left(Precision^{*}Recall\right)} \tag{8}$$

Results

The dataset considered for the current research is UNSW-NB15[5], which has 175,341 records and is also imbalanced with a higher proportion of attack samples than normal traffic. To deal with the problem of class imbalance in the UNSW-NB15 dataset, the Synthetic Minority Oversampling Technique (SMOTE) is used in the pre-processing step. This method artificially increased the minority class by creating new realistic examples of the points that already existed so that the class distribution was balanced before the training of the model. Moreover, binary classification models are used on this dataset to evaluate the different metrics of the model. The binary classification model has two classes: Class 0 for no attack and Class 1 for attack. The dataset selected for training was 70%, and for testing was 30%.

The Pseudo-algorithmic of the proposed model is given below.

Content courtesy of Springer Nature, terms of use apply. Rights reserved

Input:
- X_train (feature matrix of training data)
- y_train (one-hot encoded labels)
- input_dim = 8 (number of input features)

Output:
- Compiled and trained classification model (model)

BEGIN

1. **Initialize the model:**
   model ← Sequential()
2. **Add input and hidden layers:**
   model.add(Dense(units = 400, input_dim = 8, activation = 'relu'), Regularizer=L2)
   model.add(Dense(units = 800, activation = 'relu'))
   Dropout(0.2)
   model.add(Dense(units = 800, activation = 'relu'))
   Dropout(0.2)
   model.add(Dense(units = 400, activation = 'relu'))
3. **Add output layer:**
   model.add(Dense(units = 2, activation = 'softmax'))
4. **Compile the model:**
   optimizer ← Adam()
   loss ← categorical_crossentropy
   metrics ← ['accuracy']
   model.compile(optimizer, loss, metrics)
5. **Train the model:**
   - Set training configuration:
     batch_size ← 50
     epochs ← 100
     validation_split ← 0.33
   - Train the model and store training history:
     modelhistory ← model.fit(X_train, y_train, validation_split, batch_size, epochs)

END

Pseudo-algorithmic.

After taking the samples, an Extra Tree Classifier, which is a feature selection technique, is applied to different attributes of the dataset. The Extra Tree Classifier was not described in this research as an ordinary classifier but as a strong feature selection method that helped to select the most important features of the UNSW-NB15 data set. Since the dataset provided was of high dimension (43 features), feature reduction was extremely important to make the model not only computation-friendly but also accurate. As the Extra Tree Classifier is an ensemble-based algorithm, it measures the importance of the features against the average decrease in impurity acting upon the selected randomized decision trees. This is in order to weed out the characteristics that have little contribution to the ability to predict the model. The extra tree classifier selected only eight different attributes with the most highly relevant scores where the threshold value 0.021 was selected. All these attributes play a significant role in intrusion detection. As an example, sttl (source to destination time to live) and dttl (destination to source TTL) may indicate an anomaly in packet flow commonly related to spoofing or TTL expiration attacks. Connection-level statistics, such as the features ct_state_ttl and ct_srv_dst, are important to detect advanced patterns of misuse. The selection makes sure that only the most discriminative features are kept, which enhances the computation efficiency without substantial decreases in the accuracy of the model. The scores for this relevancy are summarized in Table 3.

S.No.	Selected attributes	High relevance values selected for attribute
1	Id	(0.1784)
2	State	(0.0372)
3	Sttl	(0.1897)
4	dttl	(0.1018)
5	dload	(0.0374)
6	swin	(0.0317)
7	ct_state_ttl	(0.0944)
8	ct_srv_dst	(0.0229)

Table 3. Selected attributes taken from the extra tree Classifier.

Content courtesy of Springer Nature, terms of use apply. Rights reserved

Epochs	Time	Loss	Accuracy	Val_accuracy
1	30s 18ms	0.1107	0.9590	0.9633
2	29s 18ms	0.0861	0.9682	0.9653
3	29s 18ms	0.0800	0.9700	0.9714
4	30s 18ms	0.0753	0.9713	0.9716
5	31s 19ms	0.0738	0.9715	0.9728

Table 4. First five selected rows of binary classification model.

Epochs	Time	Loss	Accuracy	Val_accuracy
96	28s 17ms	0.0515	0.9785	0.9783
97	28s 17ms	0.0514	0.9785	0.9800
98	29s 18ms	0.0517	0.9785	0.9813
99	32s 19ms	0.0508	0.9786	0.9804
100	31s 19ms	0.0516	0.9785	0.9818

Table 5. Last five selected rows of the binary classification model.

Class	Precision	Recall	F1-score	Support
0	0.99	0.91	0.95	16,721
1	0.96	0.99	0.98	35,882
Accuracy			97.93%	52,603
Macro Avg.	0.97	0.95	0.96	52,603
Weighted Avg	0.97	0.97	0.97	52,603

Table 6. Classification report of binary classification model.

After selecting highly relevant attributes, a standard scalar process is used which scales the selected attributes from the values 0 to 1. After applying the standardization process, the binary classification model is trained with 70% of the training data and 30% of the testing data. The model is trained with 100 epochs and a batch size of 50. As there are 100 epochs to fit the model and a description of all epochs cannot be shown in this limited space, the first five, Table 4, and the last five, Table 5, epochs are shown here.

As depicted in the last two columns of Tables 4 and 5, the trained accuracy (Accuracy) and test accuracy (Val_accuracy) are gradually increasing. While the loss values are decreasing for 0 to 99 epochs, as observed in Tables 4 and 5. The results show the true validation of the model fitting. To further confirm the strength and generalization ability of the proposed Sequential Deep Neural Network (DNN) model, the UNSW-NB15 data set was further subjected to 5-fold cross-validation. The findings revealed a standard deviation of (0.5 +/-) of the mean classification accuracy of 97.2%, showing that the model performed similarly on different data splits.

The results of binary classification in terms of accuracy, precision, recall, and f1-score are summarized in Table 6.

The table displays the findings taken from various evaluation matrices, which are required to verify the model's performance. The model performs well in terms of 97.93% accuracy.

The graph of the classification report is presented in Fig. 4.

The classification report metrics of a binary classification problem with precision, recall and F1 score of Class 0 and Class 1. Values are shown for each metric in both classes, and high performance in all metrics is shown for both classes.

Figures 5 and 6 present the learning curve accuracy and their loss drawn from Tables 4 and 5 for the binary classification model. The figure shows that as the no of epoch's increases, training and testing accuracy increases, and their loss is decreased.

To further validate the results, confusion matrices were used to classify attacks according to binary class, which is shown in Fig. 7.

Plot 4 shows the accuracy of a model during training and testing over 100 epochs. The blue line represents the training accuracy, and the orange line represents the testing accuracy. Both accuracies increase steadily, with the training accuracy reaching a higher value compared to the testing accuracy, which fluctuates somewhat during the training process.

Plot 5 shows the loss of a binary classification model for Network Intrusion Detection Systems (NIDS) over 100 epochs. The blue line represents the training loss, which decreases steadily, while the orange line represents the testing loss, which fluctuates but also tends to decrease. Both losses stabilize towards the end of the training process.

Content courtesy of Springer Nature, terms of use apply. Rights reserved

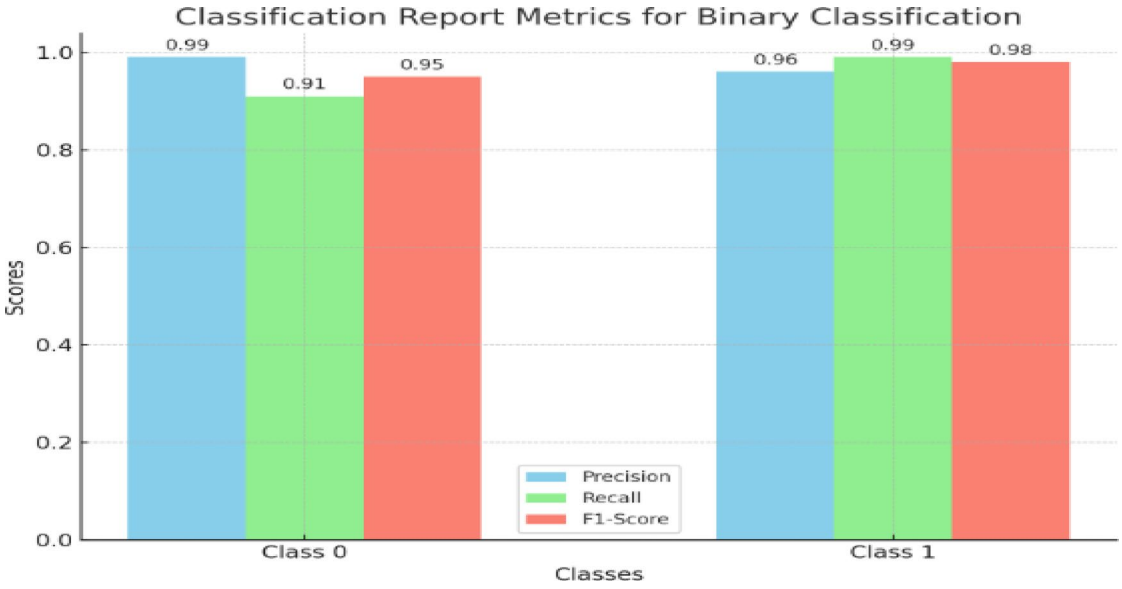


Fig. 4. Graph of classification report.

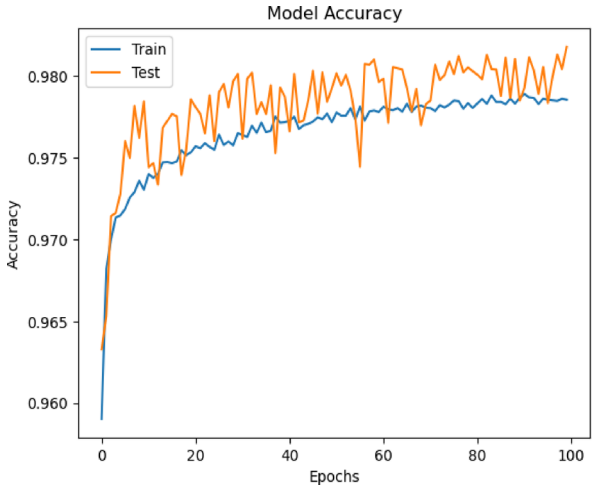


Fig. 5. Learning curve accuracy.

A total of 56,203 test data points were chosen to evaluate the sequential deep learning model. The model correctly predicts 35,666 values for Class 1 and 15,255 values for Class 0. For the understanding of the class distinction, ROC curves of the binary classification model were plotted in Fig. 8. With a variety of threshold values obtained from the probable outcomes of sequential deep learning models, a ROC curve shows the true class rate against the false class rate. The true class rate indicates the likelihood of correctly classifying attacks in contrast to the false class rate, which indicates the possibility of false alarms, which occur when an attack is incorrectly classified.

The confusion matrix shows in Fig. 7 a high number of correct predictions (15255 for class 0 and 35666 for class 1), with relatively fewer misclassifications (1466 false positives and 216 false negatives).Figure 7 shows the model's performance at identifying normal versus malicious traffic through the ROC curve. A high Area under the Curve value in ROC indicates how well the model detects intrusions. This performance achieves an effective relationship between false negatives (missed attacks) and false positives (flagging legitimate traffic). Manufacturers can incorporate this model into their real-time network monitoring programs that operate in critical industries, including banking and healthcare, to minimize financial and privacy exposure from security breaches. The reliable performance of the ROC curve generates confidence in the operational stability of the system.

The ROC curve shows the performance of a binary classification model for both classes, with the true class rate plotted against the false class rate. Both Class 0 and Class 1 curves have an area under the curve (AUC) of 0.99, indicating excellent model performance with high true positive rates and low false positive rates.

Content courtesy of Springer Nature, terms of use apply. Rights reserved

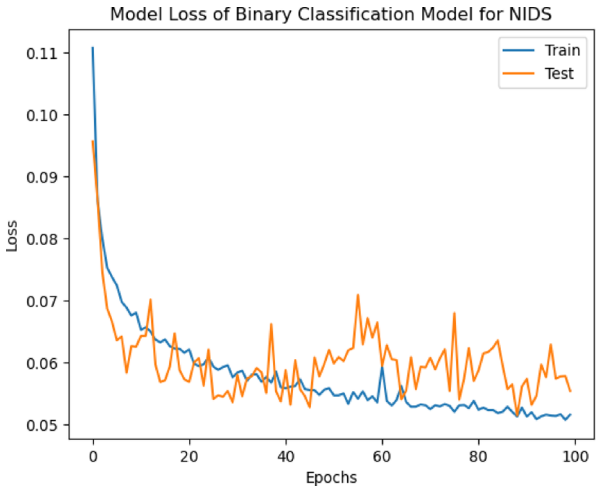


Fig. 6. Model loss.

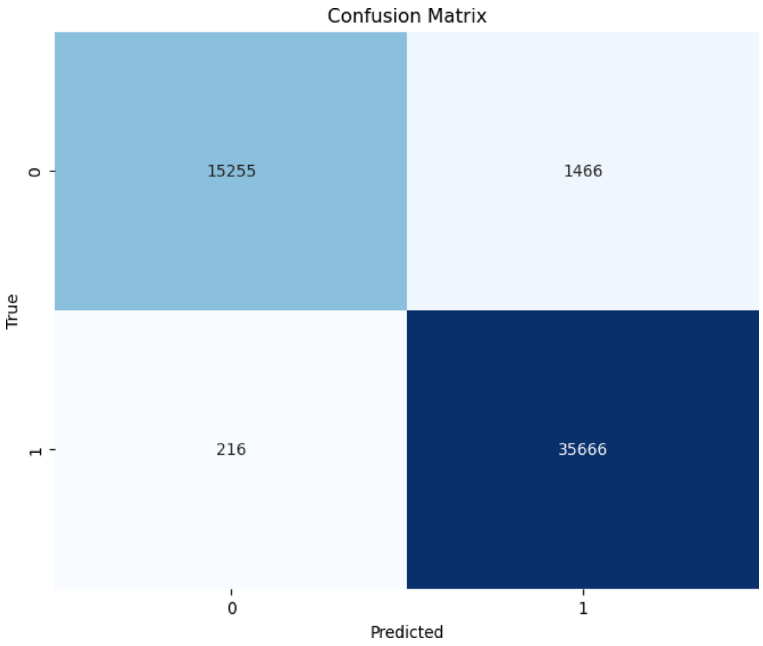


Fig. 7. Confusion matrix for binary classification model.

Comparison with benchmarks

The binary classification models are compared with the benchmark studies[6,7,37,38,44,45]. Table 7 compares the results of the current study with benchmark cases based on the accuracy score for each model case. The comparison shows that the DNN approach based on the extra tree classifier showed high accuracy in contrast to the other deep learning models.

The comparison of the different models with their accuracy is presented in Fig. 8.

The results of the proposed model, according to Table 7, suggest that the model is the most effective method of intrusion detection. Compared to previous studies (Table 7), the proposed model performed well with the highest accuracy of 97.93% using the DNN model with eight features selected by the Extra Tree Classifier. Moreover, the summarized studies in Table 7 used the sigmoid-based activation function Neural Network Model as compared to the current study, which used the ReLU-based activation function for the DNN model. Hence, the ReLU-based activation function outperforms the other-based activation functions based on the accuracy values.

Discussion

The proposed Sequential Deep Neural Network (DNN) system proved its worth by identifying network intrusions with 97.93% accuracy using a binary classification framework. The model results proved robust and

Content courtesy of Springer Nature, terms of use apply. Rights reserved

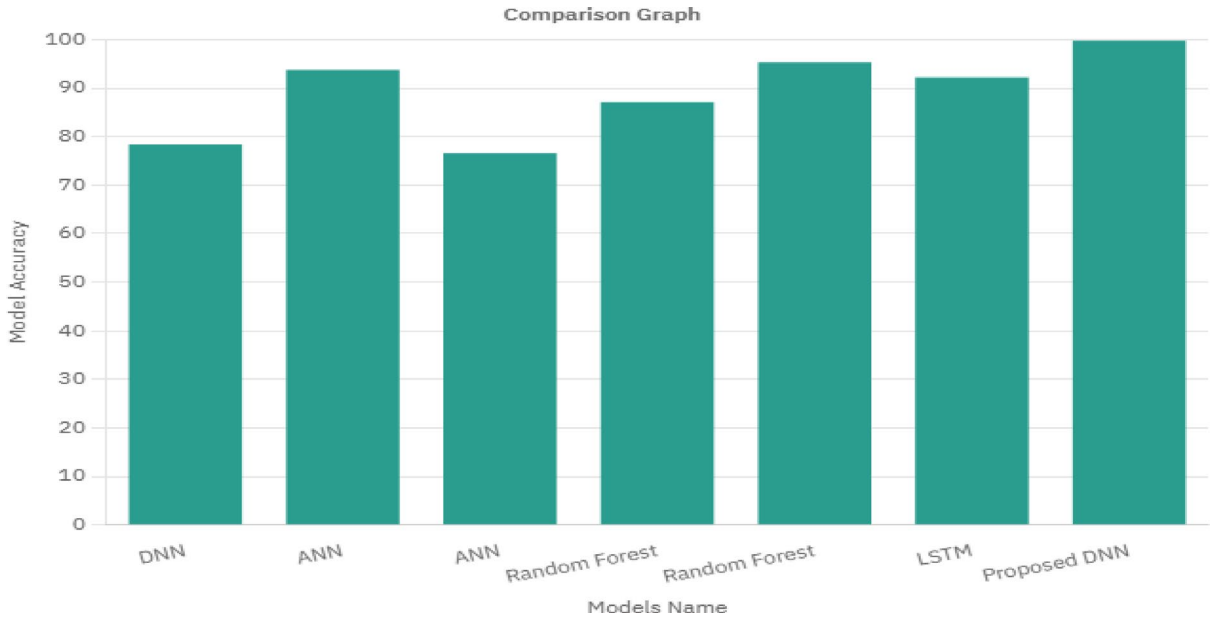


Fig. 8. ROC curve for binary classification model.

References	Model & features	Accuracy (%)	Precision (%)	Recall (%)	F1-score (%)
[44]	DNN (43)	78.4	94	73	82
[37]	ANN (19)	93.75	78.56	92	87.42
[45]	ANN (43)	79.96	89	65	76
[38]	Random forest(43)	75.38	83.30	75.38	77.51
[6]	Random forest (43)	86.	89.5	86	86
[7]	LSTM(43)	92.2	96	80	87
[46]	ANN(43)	94	N/A	N/A	N/A
[31]	Optimized CNN-LSTM (42)	78.47	78	86	82
[30]	CNN-LSTM (43)	87	85	88	86
Proposed method (sequential deep neural network)	DNN (8)	97.93	97	97	97

Table 7. Comparison of the proposed model with similar studies.

reliable based on precision, recall, and F1-score performance measurement. The approach used in this study shows better performance than Random Forest, LSTM, and ANNs since it uses L2 regularization, dropout, and ReLU activation to solve overfitting and vanishing gradient problems. The model's performance improved when the Extra Tree Classifier was used to find eight optimal attributes that reduced processing time without dropping accuracy.

The study managed missing data and scaling issues successfully by normalizing essential pre-processing actions. Testing results from confusion matrices and ROC curves reveal the model's reliability through its high AUC, which demonstrates attack versus normal traffic discrimination. The suggested study showed that the model kept performing better in each training iteration while maintaining stable accuracy and loss trends throughout epochs.

The proposed model shows superior performance through a comparative evaluation against benchmark studies. The existing approach solves the previous limitations of unsuitable class imbalance methods, specialized dataset dependency, and inadequate feature selection criteria. Application of the latest UNSW-NB15 dataset helped make the model more relevant because this dataset provides more accurate representations of actual network traffic behavior than KDD-99.

Conclusion

DNN-based models were used in this study to identify network intrusions. The effectiveness of the suggested models was assessed and compared using the UNSW-NB15 dataset. To determine the most important attributes, the suggested models use a variety of pre-processing methods, such as feature selection and data standardization. The results showed that the accuracy of the model was improved by the Extra Tree Classifier's choice of feature selection technique. According to test results, the classification models' performance on the UNSW-NB15

Content courtesy of Springer Nature, terms of use apply. Rights reserved

dataset was satisfactory in terms of F1-score metrics, recall, accuracy, and precision. The model outperformed the results of other studies with a binary classification accuracy of 97.93%. The accuracy of the results was demonstrated by the ROC curves and confusion matrices for every class.

Future work

The proposed system provided outstanding results, but additional improvements require attention to reach maximum potential. The model needs expansion to support multiclass classification since it would allow differentiating among network attacks to deliver improved detection capabilities. A comparison of model performance optimization should be evaluated through the application of Recursive Feature Elimination (RFE) and Principal Component Analysis (PCA) as advanced feature selection techniques. The investigation of ensemble learning approaches such as bagging or boosting seeks both accuracy enhancement and decreased false positives during classification tasks. The implementation of transfer learning with pre-trained deep learning models applied to network intrusion detection domains offers both time savings during training and better performance results. Real-time deployment of the system is essential for evaluating its performance rather than artificial scenarios because it allows testing under natural network modifications and altered traffic patterns. The model would work better if we tested different Swish and GELU activation functions alongside AdamW optimization algorithm options. Improved management of these factors will help our system work better at stopping network intrusions. Moreover, the model explainability strategies will be integrated to improve prediction interpretability that allows cybersecurity experts to comprehend the prediction and develop faith in system decisions. Adversarial robustness should also be taken into consideration in the future work to fortify the model against advanced evasion attacks. Lastly, the system could be made more scalable and applicable in the real-world situation by applying federated learning methods to guarantee the privacy and security of data and allowing collaborative education within the decentralized network scenario.

Data availability

The data used to support the findings of this study are available from the corresponding authors upon request.

Received: 12 April 2025; Accepted: 23 June 2025
Published online: 15 July 2025

References

1. Olayemi, A. A. A machine learning approach for information system security. *Int. J. Inf. Comput. Secur.* **16**(12), 91–101 (2018).
2. Alghamdi, M. I. Survey on applications of deep learning and machine learning techniques for cyber security. *Int J. Interact. Mob. Technol.* 210–224, (2020).
3. Fung, C. Collaborative intrusion detection networks and insider attacks. *J. Wirel. Mob. Netw. Ubiquitous Comput. Dependable Appl.* **2**(1), 63–74 (2011).
4. Shashank, K. & Balachandra, M. Review on Network Intrusion Detection Techniques using Machine Learning. in *2018 IEEE Distrib. Comput. VLSI, Electr. Circuits Robot. Discov. 2018 - Proc.* 104–109 (2019). https://doi.org/10.1109/DISCOVER.2018.8673974.
5. Moustafa, N., Slay, J. UNSW-NB15: A comprehensive data set for network intrusion detection systems (UNSW-NB15 network data set). in *Mil. Commun. Inf. Syst. Conf. MilCIS 2015 - Proc.* (2015). https://doi.org/10.1109/MilCIS.2015.7348942.
6. Sharma, N., Yadav, N. S. & Sharma, S. Classification of UNSW-NB15 dataset using exploratory data analysis using ensemble learning. *EAI Endorsed Trans. Ind. Networks Intell. Syst.* **8**, 1–10. https://doi.org/10.4108/EAI.13-10-2021.171319 (2021).
7. Alsharaiah, M. A. et al. An innovative network intrusion detection system (NIDS): Hierarchical deep learning model based on Unsw-Nb15 dataset. *Int. J. Data Netw. Sci.* **8** (2), 709–722. https://doi.org/10.5267/j.ijdns.2024.1.007 (2024).
8. Gao, N., Gao, L., Gao, Q. & Wang, H. An intrusion detection model based on deep belief networks. in *Proc. – 2014 2nd Int. Conf. Adv. Cloud Big Data, CBD 2014* 247–252. https://doi.org/10.1109/CBD.2014.41 (2015).
9. Dong, Y., Wang, R. & He, J. Real-time network intrusion detection system based on deep learning. *Proc. IEEE Int. Conf. Softw. Eng. Serv. Sci. ICSESS* 1–4, (2019). https://doi.org/10.1109/ICSESS47205.2019.9040718
10. Alrawashdeh, K. & Purdy, C. Toward an online anomaly intrusion detection system based on deep learning. in *Proc. – 2016 15th IEEE Int. Conf. Mach. Learn. Appl. ICMLA 2016* 195–200 (2017). https://doi.org/10.1109/ICMLA.2016.167.
11. Rahul, V. K., Vinayakumar, R., Soman, K. & Poornachandran, P. Evaluating Shallow and Deep Neural Networks for Network Intrusion Detection Systems in Cyber Security. in *9th Int. Conf. Comput. Commun. Netw. Technol. ICCCNT 2018* 1–6 (2018). https://doi.org/10.1109/ICCCNT.2018.8494096.
12. Moustafa, N. & Slay, J. The evaluation of network anomaly detection systems: statistical analysis of the UNSW-NB15 data set and the comparison with the KDD99 data set. *Inf. Secur. J.* **25**, 1–3. https://doi.org/10.1080/19393555.2015.1125974 (2016).
13. Kim, T. & Pak, W. Deep learning-based network intrusion detection using multiple image transformers. *Appl. Sci.* **13**(5). https://doi.org/10.3390/app13052754 (2023).
14. Baig, M. M., Awais, M. M. & El-Alfy, E. S. M. A multiclass cascade of artificial neural network for network intrusion detection. *J. Intell. Fuzzy Syst.* **32**(4), 2875–2883. https://doi.org/10.3233/JIFS-169230 (2017).
15. Vinayakumar, R. et al. Deep learning approach for intelligent intrusion detection system. *IEEE Access* **7**, 41525–41550. https://doi.org/10.1109/ACCESS.2019.2895334 (2019).
16. Belouch, M., El, S. & Idhammad, M. A Two-Stage classifier approach using RepTree algorithm for network intrusion detection. *Int. J. Adv. Comput. Sci. Appl.* **8**(6), 389–394. https://doi.org/10.14569/ijacsa.2017.080651 (2017).
17. Sai, N. R. Network intrusion detection using a step-based deep learning approach **130002** (2023).
18. Ahmad, I., Basheri, M., Iqbal, M. J. & Rahim, A. Performance comparison of support vector machine, random forest, and extreme learning machine for intrusion detection. *IEEE Access.* **6**, 33789–33795. https://doi.org/10.1109/ACCESS.2018.2841987 (2018).
19. Peng, K. et al. Intrusion detection system based on decision tree over big data in fog environment. *Wirel. Commun. Mob. Comput.* **2018** 1–11 (2018). https://doi.org/10.1155/2018/4680867.
20. Liu, H. & Lang, B. Machine learning and deep learning methods for intrusion detection systems: A survey. *Appl. Sci.* **9**(20), 1–28. https://doi.org/10.3390/app9204396 (2019).
21. Qiao, C., Zeng, Y., Lu, H., Liu, Y. & Tian, Z. An efficient incentive mechanism for federated learning in vehicular networks. *IEEE Netw.* https://doi.org/10.1109/MNET.2023.3329366 (2023).
22. Lv, Y., Shi, W., Zhang, W., Lu, H. & Tian, Z. Do not trust the clouds easily: The insecurity of content security policy based on object storage. *IEEE Internet Things J.* **10**(12), 10462–10470. https://doi.org/10.1109/JIOT.2023.3238658 (2023).

Content courtesy of Springer Nature, terms of use apply. Rights reserved

23. Lu, H. et al. AutoD: Intelligent blockchain application unpacking based on JNI layer deception call. *IEEE Netw.* **35**(2), 215–221. https://doi.org/10.1109/MNET.011.2000467 (2021).
24. Lu, H. et al. DeepAutoD: Research on distributed machine learning oriented scalable mobile communication security unpacking system. *IEEE Trans. Netw. Sci. Eng.* **9**(4), 2052–2065. https://doi.org/10.1109/TNSE.2021.3100750 (2022).
25. Tan, Y., Huang, W., You, Y., Su, S. & Lu, H. Recognizing BGP communities based on graph neural network. *IEEE Netw.* https://doi.org/10.1109/MNET.2024.3414113 (2024).
26. Gu, Z. et al. Gradient shielding: Towards understanding vulnerability of deep neural networks, *IEEE Trans. Netw. Sci. Eng.* **8**(2) 921–932. https://doi.org/10.1109/TNSE.2020.2996738 (2021).
27. Makhdoomi, P. M. S. et al. Jan., Network-based intrusion detection: A comparative analysis of machine learning approaches for improved security. *J Cyber Secur. Technol.* (2025).
28. Alrayes, F. S., Zakariah, M., Amin, S. U., Khan, Z. I. & Alqurni, J. S. Network security enhanced with deep neural Network-Based intrusion detection system. *Comput. Mater. Contin.* **80**(1), 1457. https://doi.org/10.32604/CMC.2024.051996 (2024).
29. Ul, E., Qazi, H., Faheem, M. H. & Zia, T. applied sciences HDLNIDS: Hybrid Deep-Learning-Based Network Intrusion Detection System (2023).
30. Chen, Y. Framework design of network intrusion detection based on convolutional neural networks. *Procedia Comput. Sci.* **261**, 1356–1362. https://doi.org/10.1016/j.procs.2025.05.013 (2025).
31. Thaljaoui, A. Intelligent network intrusion detection system using optimized deep CNN-LSTM with UNSW-NB15. *Int. J. Inf. Technol.* https://doi.org/10.1007/s41870-025-02416-0 (2025).
32. Khan, F. A., Gumaei, A., Derhab, A. & Hussain, A. A novel two-stage deep learning model for efficient network intrusion detection. *IEEE Access* **7**, 14–28. https://doi.org/10.1109/ACCESS.2019.2899721 (2020).
33. Meftah, S., Rachidi, T. & Assem, N. Network based intrusion detection using the UNSW-NB15 dataset. *Int. J. Comput. Digit. Syst.* **8**(5), 477–487. https://doi.org/10.12785/ijcds/080505 (2019).
34. Sstla, V., Kolli, V. K. K., Voggu, L. K., Bhavanam, R. & Vallabhasoyula, S. Predictive model for network intrusion detection system using deep learning. *Rev. D'Intell. Artif.* **34**(3), 323–330. https://doi.org/10.18280/ria.340310 (2020).
35. Rawat, S., Srinivasan, A., Ravi, V. & Ghosh, U. Intrusion detection systems using classical machine learning techniques vs integrated unsupervised feature learning and deep neural network. *Internet Technol. Lett.* **5** (1), 1–5. https://doi.org/10.1002/itl2.232 (2022).
36. Koroniotis, N., Moustafa, N., Sitnikova, E. & Slay, J. Towards developing network forensic mechanism for botnet activities in the IoT based on machine learning techniques. *Lect Notes Inst. Comput. Sci. Soc. Telecommun Eng. LNICST*. **235**, 30–44. https://doi.org/10.1007/978-3-319-90775-8_3 (2018).
37. Kasongo, S. M. & Sun, Y. Performance analysis of intrusion detection systems using a feature selection method on the UNSW-NB15 dataset. *J. Big Data*. **7** (1), 1–20. https://doi.org/10.1186/s40537-020-00379-6 (2020).
38. Amarouche, K. Machine and deep learning-based intrusion detection and comparison in & internet of things. *J. Nav. Sci. Eng.* **18**(2017), 2003–2005 (2022).
39. Hameed, A. & Bawany, N. Z. Network intrusion detection using oversampling technique and machine learning algorithms. *PeerJ Comput. Sci.* **8**, 1–19. https://doi.org/10.7717/peerj-cs.820 (2022).
40. Vinayakumar, R. et al. Deep learning approach for intelligent intrusion detection system. *IEEE Access* **7**, 41525–41550. https://doi.org/10.1109/ACCESS.2019.2895334 (2019).
41. Lin, P., Ye, K. & Xu, C. Z. Dynamic network anomaly detection system by using deep learning techniques. *Lect Notes Comput. Sci. (including Subser. Lect Notes Artif. Intell. Lect Notes Bioinform.)* **11513**, 161–176. https://doi.org/10.1007/978-3-030-23502-4_12 (2019).
42. Ludwig, S. A. Intrusion detection of multiple attack classes using a deep neural net ensemble. *2017 IEEE Symp. Ser. Comput. Intell. SSCI 2017 - Proc.* **2018**, 1–7. https://doi.org/10.1109/SSCI.2017.8280825 (2018).
43. Niyaz, Q., Sun, W., Javaid, A. Y. & Alam, M. A deep learning approach for network intrusion detection system. *EAI Int. Conf. Bio-inspired Inf. Commun. Technol.* https://doi.org/10.4108/eai.3-12-2015.2262516 (2015).
44. Vinayakumar, R., Alazab, M., Member, S. & Soman, K. P. Deep learning approach for intelligent intrusion detection system. *IEEE Access* **7**, 41525–41550. https://doi.org/10.1109/ACCESS.2019.2895334 (2019).
45. Mebawondu, J. O., Alowolodu, O. D., Mebawondu, J. O. & Adetunmbi, A. O. Network intrusion detection system using supervised learning paradigm. *Sci. Afr.* **9**, 11–21. https://doi.org/10.1016/j.sciaf.2020.e00497 (2020).
46. Arun, C. B. et al. Enhancing Network Intrusion Detection using Artificial Neural Networks: An Analysis of the UNSW-NB15 dataset. in *2nd IEEE Int. Conf. Integr. Intell. Commun. Syst. ICIICS 2024* (2024). https://doi.org/10.1109/ICIICS63763.2024.10859396.

Acknowledgements

Princess Nourah bint Abdulrahman University Researchers Supporting Project number (PNURSP2025R845), Princess Nourah bint Abdulrahman University, Riyadh, Saudi Arabia.

Author contributions

Muhammd Amir Khan and Muhammad Farhan perform the Original Writing Part, Software, and Methodology; Hafiz Waheed-ud-din and Tehseen Mazhar perform Rewriting, investigation, design Methodology, and Conceptualization; Saadat Ullah, Umar Farooq Khattak and, Ines Hilali Jaghdam perform the related work part and manage results and discussions; Tehseen Mazhar Muhammd Amir Khan and Muhammad Farhan perform related work part and manage results and discussion; Saadat Ullah, Tehseen Mazhar and Muhammd Amir Khan perform Rewriting, design Methodology, and Visualization; Ines Hilali Jaghdam and Umar Farooq Khattak performs Rewriting, design Methodology, and Visualization.

Funding

Princess Nourah bint Abdulrahman University Researchers Supporting Project number (PNURSP2025R845), Princess Nourah bint Abdulrahman University, Riyadh, Saudi Arabia.

Declarations

Competing interests

The authors declare no competing interests.

Content courtesy of Springer Nature, terms of use apply. Rights reserved

Additional information

Correspondence and requests for materials should be addressed to M.A.K. or T.M.

Reprints and permissions information is available at www.nature.com/reprints.

Publisher's note Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.

Open Access This article is licensed under a Creative Commons Attribution-NonCommercial-NoDerivatives 4.0 International License, which permits any non-commercial use, sharing, distribution and reproduction in any medium or format, as long as you give appropriate credit to the original author(s) and the source, provide a link to the Creative Commons licence, and indicate if you modified the licensed material. You do not have permission under this licence to share adapted material derived from this article or parts of it. The images or other third party material in this article are included in the article's Creative Commons licence, unless indicated otherwise in a credit line to the material. If material is not included in the article's Creative Commons licence and your intended use is not permitted by statutory regulation or exceeds the permitted use, you will need to obtain permission directly from the copyright holder. To view a copy of this licence, visit http://creativecommons.org/licenses/by-nc-nd/4.0/.

© The Author(s) 2025

Content courtesy of Springer Nature, terms of use apply. Rights reserved

Terms and Conditions

Springer Nature journal content, brought to you courtesy of Springer Nature Customer Service Center GmbH (“Springer Nature”).
Springer Nature supports a reasonable amount of sharing of research papers by authors, subscribers and authorised users (“Users”), for small-scale personal, non-commercial use provided that all copyright, trade and service marks and other proprietary notices are maintained. By accessing, sharing, receiving or otherwise using the Springer Nature journal content you agree to these terms of use (“Terms”). For these purposes, Springer Nature considers academic use (by researchers and students) to be non-commercial.
These Terms are supplementary and will apply in addition to any applicable website terms and conditions, a relevant site licence or a personal subscription. These Terms will prevail over any conflict or ambiguity with regards to the relevant terms, a site licence or a personal subscription (to the extent of the conflict or ambiguity only). For Creative Commons-licensed articles, the terms of the Creative Commons license used will apply.
We collect and use personal data to provide access to the Springer Nature journal content. We may also use these personal data internally within ResearchGate and Springer Nature and as agreed share it, in an anonymised way, for purposes of tracking, analysis and reporting. We will not otherwise disclose your personal data outside the ResearchGate or the Springer Nature group of companies unless we have your permission as detailed in the Privacy Policy.
While Users may use the Springer Nature journal content for small scale, personal non-commercial use, it is important to note that Users may not:

1. use such content for the purpose of providing other users with access on a regular or large scale basis or as a means to circumvent access control;
2. use such content where to do so would be considered a criminal or statutory offence in any jurisdiction, or gives rise to civil liability, or is otherwise unlawful;
3. falsely or misleadingly imply or suggest endorsement, approval , sponsorship, or association unless explicitly agreed to by Springer Nature in writing;
4. use bots or other automated methods to access the content or redirect messages
5. override any security feature or exclusionary protocol; or
6. share the content in order to create substitute for Springer Nature products or services or a systematic database of Springer Nature journal content.

In line with the restriction against commercial use, Springer Nature does not permit the creation of a product or service that creates revenue, royalties, rent or income from our content or its inclusion as part of a paid for service or for other commercial gain. Springer Nature journal content cannot be used for inter-library loans and librarians may not upload Springer Nature journal content on a large scale into their, or any other, institutional repository.
These terms of use are reviewed regularly and may be amended at any time. Springer Nature is not obligated to publish any information or content on this website and may remove it or features or functionality at our sole discretion, at any time with or without notice. Springer Nature may revoke this licence to you at any time and remove access to any copies of the Springer Nature journal content which have been saved.
To the fullest extent permitted by law, Springer Nature makes no warranties, representations or guarantees to Users, either express or implied with respect to the Springer nature journal content and all parties disclaim and waive any implied warranties or warranties imposed by law, including merchantability or fitness for any particular purpose.
Please note that these rights do not automatically extend to content, data or other material published by Springer Nature that may be licensed from third parties.
If you would like to use or distribute our Springer Nature journal content to a wider audience or on a regular basis or in any other manner not expressly permitted by these Terms, please contact Springer Nature at

onlineservice@springernature.com